AI Engineer (Data)

Onboarding guide to Labs 1 to 6. How to get the labs, how to run them, and what each one teaches. Labs 1 to 4 come in two versions, essentials and full; Labs 5 and 6 are essentials only.

1. The labs in one page

Every lab uses the same fictional company, **FreshCart**, a food delivery business in India. Its documents are two refund policies (2022 and 2024), a partner contract, a scanned procedure, a Word table, a slide deck and a spreadsheet. Its warehouse is one database file of orders, customers, refund claims and restaurants. The labs build, step by step, a system that answers questions about FreshCart from those documents and that data.

Day	Lab	What you build	Essentials	Full
1	1	Read seven documents, cut them into pieces, search them by meaning	lab01_essentials	lab01_ingest
2	2	The same pieces in three vector databases: Chroma, Qdrant and FAISS	lab02_essentials	lab02_vector_stores
3	3	Better search: keyword search, a reranker and a recency filter, scored with nDCG@10	lab03_essentials	lab03_hybrid, lab03b_real_models
4	4	An answer service with a real model that cites its sources, checked in code	lab04_essentials	lab04_rag_api
5	5	Give a model tools, watch which it picks, and serve tools over MCP	lab05_essentials	none
6	6	Questions in English to SQL, and a semantic layer for numbers you can defend	lab06_essentials	none

Two kinds of lab

- **Essentials**: short, about 50 minutes, LangChain from start to finish, a plain comment on every line of code. You click Run all and read each step while it runs. These are the course labs from Lab 5 on, and the best place to start for Labs 1 to 4.
- **Full** (Labs 1 to 4 only): longer, 75 to 80 minutes, built on the course's own code, done in pairs. You predict a number in a cell marked YOUR CALL before each result, and you hand in a written answer sheet. Use them to go deeper after the essentials, or when your trainer runs them in class.

Which to do? If you are new to these tools, do the essentials version of every lab first. If you finish early, open the full version of the same lab: it measures more and asks you to defend your choices.

2. Getting started

2.1 What you need

- A **Databricks workspace with serverless compute**. In class, your trainer gives you one. On your own, the free edition works for Labs 1 to 3.
- For Labs 4, 5 and 6, and the full Lab 3b: Databricks' **served models**. The labs call **databricks-meta-llama-3-3-70b-instruct**, a language model Databricks runs for you. You sign in as yourself, so there is no key to copy. It is billed per use, a few cents a run.
- A browser. Nothing has to be installed on your computer.

2.2 On your own workspace: get the labs from GitHub

1. In the Databricks sidebar, click **Workspace**, then open your **Home** folder.
2. Click **Create**, then **Git folder**.
3. Paste **https://github.com/mogupta84/ai-engineer-data-labs** and click **Create Git folder**.
4. The essentials are in **databricks/essentials/**. The full labs are in **databricks/**.

Each notebook downloads the documents, the warehouse and the test questions it needs on its first run. Nothing to upload.

2.3 In class: make your own copy first

The class copy of every lab is in **Workspace, freshcart-labs, freshcart_notebooks**. You can open and read it, but you run your own copy, so your results never mix with anyone else's. If you run the class copy, its first cell stops with *This is the class copy, which you cannot run*.

1. In **Workspace**, open **freshcart-labs**.
2. Click the three dots beside **freshcart_notebooks** and choose **Clone**.
3. Pick your **Home** folder and confirm.
4. Open the lab from your copy in your Home folder, never from freshcart-labs.

2.4 Running a notebook

1. Open the notebook and attach **Serverless** from the compute menu at the top right.
2. Click **Run all**.
3. The first cell installs the libraries and **restarts Python**. That takes one to two minutes and is expected.
4. Read each step while it runs. Every step says **What this does** and **What you should see**. Compare your output with it.
5. At the end, the **Look it up yourself** cell lists every setting of the libraries the lab used, and **Your notes** is a cell for your own answers.

Numbers can move a little between runs. A language model does not always word an answer the same way, even at its most predictable setting. Where that happens, What you should see gives the range we measured, for example "14 or 15 of 16".

3. Lab 1: read, cut and search the documents

Turn seven files in four formats (PDF, Word, PowerPoint, Excel) into labelled pieces you can search by meaning, and find the problems that no error message tells you about.

Essentials notebook: lab01_essentials (about 50 minutes)

- Load the PDFs and find the one that comes back empty: the scanned procedure is a picture of text.
- Load the Word, PowerPoint and Excel files, and check what each loader kept.
- Cut the documents into pieces of about 500 characters, and label every piece with its file and date.
- Turn the pieces into embeddings (lists of numbers that capture meaning) and search them.
- Search again using the labels.

What you will find

- The Word table loses its column names, and in the Excel file the title row is read as the header, so the columns come out as "Unnamed".
- The withdrawn 2022 policy beats the current one: 0.877 against 0.791 for the same question. Labels fix it.
- A question the documents cannot answer, about Singapore, still gets results, scoring 0.824: higher than a question that has an answer.

Full lab: lab01_ingest (about 75 minutes, in pairs)

Five formats, two that fail silently, three loader libraries compared, two tools that read page layout, and five real support questions scored.

4. Lab 2: the same pieces in three vector databases

A vector database stores embeddings and finds the closest ones fast. Put the same pieces into three of them and compare what they return.

Essentials notebook: lab02_essentials (about 45 minutes)

- Build the answer key with an exact search over every piece.
- Load the same pieces into Chroma, Qdrant and FAISS, and ask all three the same question.
- Score them on ten questions for recall (how many of the right pieces they find) and time.
- Write the same filter three ways, and check what filtering after the search does.

What you will find

- For the same question the three report scores on three different scales: 0.877, 0.123 and 0.246. A score only means something inside one database.
- All three reach recall@3 of 1.00 on this small set.
- Filtering after the search quietly returns 2 results instead of 3.

Full lab: lab02_vector_stores (about 80 minutes, in pairs)

Scales up to where speed differs, turns the speed against recall setting (about 0.97 recall at ef 40), measures compression and filters at scale, and ends with a memo recommending one database.

5. Lab 3: better search, and a score that says what helped

Add one stage at a time to meaning search, and score each stage on 20 judged questions with nDCG@10: a score from 0 to 1 that rewards putting the right pieces near the top of the first 10.

Essentials notebook: lab03_essentials (about 50 minutes)

- Learn how nDCG@10 is calculated from the judged questions.
- Stage 1: meaning search. Stage 2: add keyword search, merged by rank.
- Stage 3: add a reranker, a model that reorders the shortlist.
- Stage 4: add a recency filter that drops withdrawn documents.
- Put all stages side by side and find the questions that got worse.

What you will find

- Each stage raises the score: 0.754 meaning only, 0.804 merged with keywords, 0.865 reranked, 0.917 with the recency filter. Keyword search alone scores 0.774.
- The average hides losses: 4 questions get worse along the way.
- The reranker's model matters: the default one drops the score to 0.647.

Full lab: lab03_hybrid and lab03b_real_models (about 75 plus 20 minutes, in pairs)

The full Lab 3 runs the same four stages with the course's own code and follows two questions through every stage. Lab 3b repeats them with models Databricks serves, including a document reader for the scanned PDF.

6. Lab 4: an answer service that proves where its answer came from

A real language model writes the answer from the pieces Lab 3 finds, and cites the identifier of each piece it used. Code then checks every citation. This is called grounding: the model may only use what it was given.

Essentials notebook: lab04_essentials (about 50 minutes)

- Fill a prompt with the 5 best pieces, each tagged with its identifier.
- Let the model write the answer, then check every identifier in code and return six fields: answer, citations, confidence, retrieved pieces, refused, and the reason.
- Break it on purpose by adding a made-up source to every third answer.
- Measure it on 25 questions: 20 the documents can answer, 5 they cannot.

What you will find

- The check catches 3 of 3 made-up sources, although the answers read normally.
- It refuses all 5 questions it cannot answer, and 2 or 3 of the 20 it could. Close to 17 or 18 of 20 answers state the right fact.
- A stricter prompt removes the code's misreadings but makes the model refuse good questions.

Full lab: lab04_rag_api (about 80 minutes, in pairs)

Measures the same service with the course's own code: it catches 8 of 8 fake citations, refuses 1 of 20 good questions and 4 of 5 unanswerable ones, and finds the two answers the code read wrongly. It can also run as a real API on your own computer; the GitHub README explains how.

7. Lab 5: give the model tools

Tool calling means the model asks your code to run a named function, and your code sends back the result. MCP (Model Context Protocol) is a standard way to offer tools to any AI app.

Essentials notebook: lab05_essentials (about 50 minutes)

- Turn three Python functions into tools: search the policy documents, get an official business number, run a read-only SQL query.
- Ask 10 questions and see which tool the model picks for each.
- Problem 1: give the tools vague descriptions. Problem 2: ask the model to change data.
- Write the tool loop: run the tools the model asks for and send the results back.
- Problem 3: take away the official-number tool. Then serve two tools from an MCP server.

What you will find

- With good descriptions the model picks the right tool 9 or 10 times out of 10. With one-line descriptions it drops to 6 of 10, writes SQL on tables that do not exist, and invents a metric name.
- Asked to delete or change data, it sends a DELETE or UPDATE 3 times out of 3. The check in code blocks all 3.
- Without the official-number tool, its own SQL says March 2024 revenue was 590,657.18 instead of 531,043.03, and May had 491 orders instead of 450. No error appears.
- The same tools served over MCP give the same answer.

8. Lab 6: questions in English, numbers you can defend

NL2SQL (natural language to SQL) means a model turns a question into a database query. A semantic layer is a list of business numbers, each written once with its rules, so the model picks a number instead of writing new SQL.

Essentials notebook: lab06_essentials (about 50 minutes)

- Let the model write SQL for "How many orders were placed in May 2024?" and compare with the reviewed answer.
- Add FreshCart's business rules to the prompt, then four worked examples.
- Score 16 questions three ways: schema only, with rules, with rules and examples.
- Work out one phrase, refund rate, two honest ways.
- Build a semantic layer, then turn a number into a sentence and a chart, in code.

What you will find

- The model says 491 orders; the reviewed answer is 450. The extra 41 come from two test accounts. The rules alone do not fix it; the worked examples do.
- The score rises from 7 or 8 of 16 to 10 or 11 with the rules, and 14 or 15 with rules and examples.
- Refund rate is 0.0452 or 0.0409, depending on what you count.
- The semantic layer answers 6 questions, gets 4 right and refuses 10, rather than guessing. Its two mistakes come from misreading the question.

9. If something goes wrong

What you see	What it means and what to do
This is the class copy, which you cannot run	You opened the lab inside freshcart-labs. Make your own copy (section 2.3) and open it from your Home folder.
Python restarts after the first cell	Expected. The libraries just installed need a fresh start.
A red line about fastapi and starlette (Lab 5)	A warning about packages this lab does not use. Ignore it.
REQUEST_LIMIT_EXCEEDED, or a model step that is slow	The served model allows only a few calls a second for the whole workspace. The notebook waits and retries. If many people run Labs 4 to 6 at once, wait a minute and run the cell again.
The model step fails with an access or not-found error	Your workspace does not offer the served models. Labs 1 to 3 still work; Labs 4 to 6 need them.
Your number differs a little from What you should see	Normal for steps that call a model. Check it is inside the range given.
Full Lab 2, Step 7 prints *skipped*	That step reads a search service that only runs during class. The rest of the lab runs as normal.
Your notes disappeared	Write in the **Your notes** or **Your write-up** cell inside the notebook, then export the notebook. Files outside it are replaced each session.

10. Words used in the labs

Word	Meaning
Piece (chunk)	A short part of a document, about 500 characters, that is stored and searched on its own.
Embedding	A list of numbers that captures what a piece of text means, so similar meanings get similar numbers.
Vector database	A store that keeps embeddings and quickly finds the ones closest to a question's embedding.
Recall	Of the right pieces, the share a search actually found.
nDCG@10	A score from 0 to 1 for a search: high when the right pieces are near the top of the first 10.
Reranker	A model that reorders a shortlist of pieces by how well each one answers the question.
Grounding	The model may only use the pieces it was given, not its memory.
Citation	The identifier of the exact piece a fact came from, written in square brackets.
Tool, tool call	A function the model may ask your code to run; the request is the tool call.
MCP	Model Context Protocol: a standard way for a program to offer tools to any AI app.
NL2SQL	A model turning a question in English into a SQL query.
Semantic layer	Business numbers, each written once with its rules, that questions are matched to.

The labs, their data and this guide: https://github.com/mogupta84/ai-engineer-data-labs